

Programování 2

12. cvičení, 5-05-2026

Farní oznamy

1. Tento text a kódy ke cvičení najdete v repozitáři cvičení na <https://github.com/PKvasnick/Programovani-2>.
2. **Domácí úkoly:** Domino a cesta věže. Promluvíme podrobněji.*
3. **Zápočtové programy:** pořad nemám témata od čtyřech z vás, prosím ozvěte se. Pokud jste se ozvali a ztratili se v šumu, ozvěte se ještě jednou a třeba hlasitěji.

Meno	E-mail	Body za DU 30/4/26	Téma zápočtového programu	Stav
Petr Beneš	peta.benes@outlook.com	190		
Adam Celecký	Adam.celecky@gmail.com	172		
Petr Folwarczny	petr.folwarczny@seznam.cz	170	Matice, min-plus	V jednání
Jáchym Galuška	jachymgaluska92@gmail.com	171	Bakterie s NN	Dohodnuté
Vojtěch Horký	vojtech.horky200@student.cuni.cz	181	Piškvorky	V jednání
Jana Janoušová	janousova-jana@seznam.cz	190	Křížovky	Dohodnuté
Jan Kodeš	honza.hk6@gmail.com	138	Matice	Domluveno
Anežka Marešová	anezkamaresovaa@gmail.com	155	Matice	Domluveno
Tereza Matějková	tematka@email.cz	131	Lodě	Dohodnuté
Jakub Moc	JakubMoc@email.cz	180		
Robin Novotný	robin.novotny25@seznam.cz	171	Piškvorky	Dohodnuté
Luis Ondřej	luis@centrum.cz	166		
Pavel Petrok	pavel.petrok220@student.cuni.cz	190	Piškvorky	Dohodnuté
Daniel Ransdorf	school.ransdorf@proton.me	192	Server časových radov	Odovzdané
Jakub Remeš	remesjakub4@gmail.com	160	Piškvorky	Domluveno
Dominik Martin Rigász	d.m.rigaszi@gmail.com	0		
Richard Smutný	smurichrd@gmail.com	121	Tetri	Dohodnuté
Filip Valenta	filip.valenta001@gmail.com	189	Formulky	Domluveno
Zhao jun Yang	yangzhaojun666@gmail.com	150	Minesweeper	Domluveno

Dnešní program:

- Domácí úkoly
- Poznámka k topologickému třídění
- Piškvorky

Domácí úkoly

Cesta věže

Toto je malý problém, takže není vůbec potřebné moc se zamýšlet. Prozkoumáme všechny možné tahy věže, a na každém navštíveném poli necháme informaci o tom, kolika nejméně tahy se k němu lze dostat.

Jde o prohledávání do šířky, BFS: končíme, když máme vyplněna všechna dostupná pole, a vrátíme údaj ze zadaného cílového pole.

Abychom předešli zmnožování kódu pro různé směry pohybu věže, uděláte si funkci, která vrátí všechna pole, dostupná z dané pozice věže.

Používáme prioritní frontu: nejdřív se zabýváme poli, které vidíme poprvé.

```
# Find the shortest path of the rook on chessboard
from itertools import product
import heapq

SIZE = 8
INF = 10_000

@lambda cls: cls() # Create class instance immediately
class Chessboard:
    """
    The chessboard contains information on how many
    moves are needed to reach a given field.
    """
    def __init__(self):
        """Just create chessboard"""
        self.chessboard = dict([(i,j),INF) for i, j in product(range(SIZE), range(SIZE))]
        self.start = None
        self.end = None

    def is_in_range(self, k, l):
        return (k,l) in self.chessboard.keys()

    def set_obstacle(self, i, j):
        self.chessboard[(i,j)] = -1

    def set_start(self, i, j):
        self.start = (i, j)
        self.chessboard[(i,j)] = 0

    def set_end(self, i, j):
        self.end = (i, j)

    def get_steps(self, i, j):
        return self.chessboard[(i,j)]

    def set_steps(self, i, j, steps):
        self.chessboard[(i,j)] = steps

    def rook_fields(self, i, j):
        """Return a list of fields controlled by a rook at (i, j)"""
        steps = [(1, 0), (0, 1), (-1, 0), (0, -1)]
        fields = []
        for s, t in steps:
```

```

        k = i + s
        l = j + t
        while self.is_in_range(k, l) and self.chessboard[k,l] != -1:
            fields.append((k, l))
            k = k + s
            l = l + t
    return fields

```

```

def print(self):
    chart = [["_"] for _ in range(SIZE)] for _ in range(SIZE)]
    for pos, steps in self.chessboard.items():
        i, j = pos
        if steps == -1:
            chart[i][j] = "x"
        elif steps == INF:
            chart[i][j] = "?"
        else:
            chart[i][j] = str(steps)
    for i in range(SIZE):
        print(*chart[i])
    print()

```

```

def read_chessboard():
    global Chessboard
    for i in range(SIZE):
        row = input().strip()
        for j in range(SIZE):
            if row[j] == ".":
                continue
            elif row[j] == "x":
                Chessboard.set_obstacle(i, j)
            elif row[j] == "v":
                Chessboard.set_start(i, j)
            elif row[j] == "c":
                Chessboard.set_end(i, j)
    return

```

```

def grade_chessboard():
    global Chessboard
    stack = []
    heapq.heappush(stack, (0, *Chessboard.start))
    while stack:
        steps, row, col = heapq.heappop(stack)
        for field in Chessboard.rook_fields(row, col):
            field_steps = Chessboard.get_steps(*field)
            if field_steps <= steps + 1:
                continue
            else:
                Chessboard.set_steps(*field, steps+1)
                heapq.heappush(stack, (steps+1, *field))
    Chessboard.print()

```

```

return

def main():
    global Chessboard
    read_chessboard()
    grade_chessboard()
    steps_to_end = Chessboard.get_steps(*Chessboard.end)
    if steps_to_end == INF:
        print(-1)
    else:
        print(steps_to_end)
    return

if __name__ == '__main__':
    main()

```

Domino

Tady máme dvě osobitosti:

- symetrii kostek - kostku AB můžeme použít jako AB i BA. Je potřeba s ní rozumně naložit tak, abychom kód zbytečně nevětvali.
- datovou strukturu: Jaká struktura bude nejlépe vyhovovat pro získání všech kostek, které můžeme přidat na konec stávajícího řetězce?
- také máme určitou jemnost v tom, že můžeme mít několik stejných kostiček, z nichž ale stačí vždy vyzkoušet jedinou.

Algoritmicky jde o čisté prohledávání do hloubky:

```

from collections import deque

max_length: int = 0

current_string: deque = deque()
current_pieces = [[0 for _ in range(39)] for _ in range(39)]

def read_data():
    global current_pieces
    numbers = list(map(int, input().split()))
    num_pieces = numbers[0]
    while len(numbers) < num_pieces * 2 + 1:
        numbers.extend(list(map(int, input().split())))
    for i in range(1, len(numbers) - 1, 2):
        current_pieces[numbers[i]][numbers[i + 1]] += 1
        current_pieces[numbers[i + 1]][numbers[i]] += 1

def solve():
    global current_pieces

```

```

global current_string
global max_length
j_final = current_string[-1][1]
for i in range(1, 39):
    if current_pieces[j_final][i] > 0:
        current_string.append((j_final, i))
        current_pieces[j_final][i] -= 1
        current_pieces[i][j_final] -= 1
        solve()
        current_pieces[j_final][i] += 1
        current_pieces[i][j_final] += 1
        if len(current_string) > max_length:
            max_length = len(current_string)
        current_string.pop()

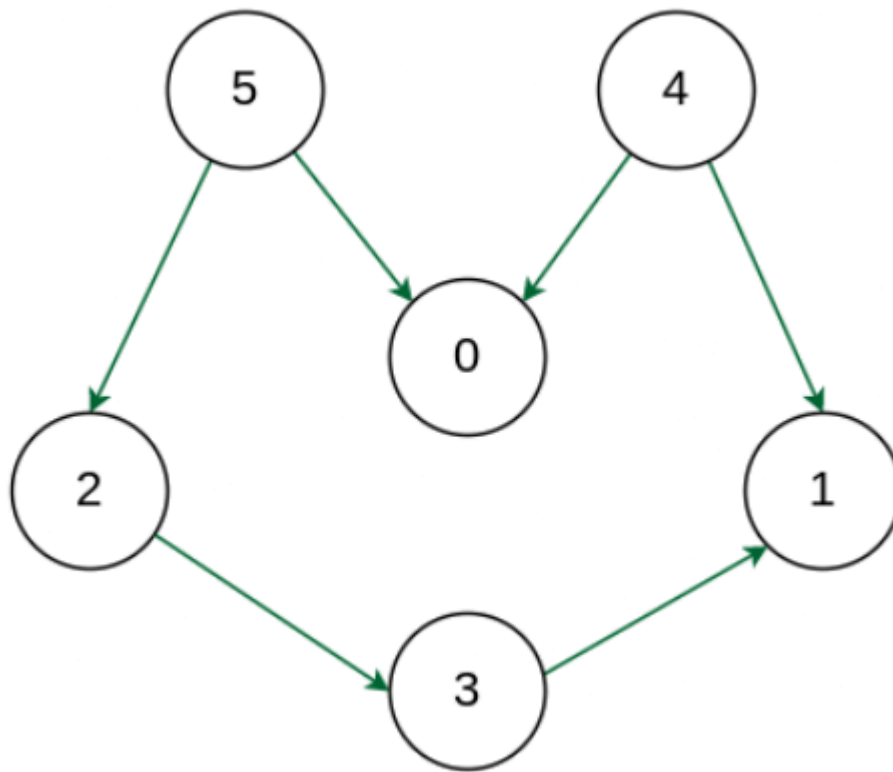
def main():
    global max_length
    global current_string
    global current_pieces
    read_data()
    # Inicializace - zkusíme začít s každou kostkou.
    for i in range(1, 39):
        for j in range(i, 39):
            # ze stejných jenom jednu
            if current_pieces[i][j] > 0:
                current_string.append((i, j))
                current_pieces[i][j] -= 1
                current_pieces[j][i] -= 1
                solve()
                current_pieces[i][j] += 1
                current_pieces[j][i] += 1
                if len(current_string) > max_length:
                    max_length = len(current_string)
                current_string.pop()
    print(max_length)

if __name__ == "__main__":
    main()

```

1. Nepěknou vlastností tohoto kódu je, že se při inicializaci opakuje kód velice podobný tomu, co dělá funkce `solve()`. Někaké nápady, jak toto vyřešit elegantně?
2. Intuitivně se nabízí topologické třídění jako cesta k řešení - jak moc dokážeme roztáhnout graf vztahů kostek? Jsou s tím dva problémy:
 - Graf vztahů kostek není DAG
 - Kombinatorický problém: pokud mám k kostek se stejnými čísly, pak každá cesta, která obsahuje tyto kostky, se vyskytne $k!$ krát, pokud si nedám pozor.

Poznámka k topologickému třídění



Toto je malý graf, a dobře funguje běžný algoritmus:

```
from collections import defaultdict, deque

class Graph:
    def __init__(self, vertices):
        self.graph = defaultdict(list)
        self.V = vertices

    def addEdge(self, u, v):
        self.graph[u].append(v)

    def dfsSort(self, v, visited, stack):
        visited[v] = True
        for i in self.graph[v]:
            if not visited[i]:
                self.dfsSort(i, visited, stack)
        stack.appendleft(v) # Add to front after exploring

    def topologicalSort(self):
        visited = [False] * self.V
        stack = deque()

        for i in range(self.V):
            if not visited[i]:
                self.dfsSort(i, visited, stack)
```

```

        print(stack)

def main():
    g = Graph(6)
    g.addEdge(5, 2)
    g.addEdge(5, 0)
    g.addEdge(4, 0)
    g.addEdge(4, 1)
    g.addEdge(2, 3)
    g.addEdge(3, 1)

    g.topologicalSort()

if __name__ == "__main__":
    main()

```

Kdybychom ale měli velký graf, nezpůsobila by nám rekurze problémy? Jak by vypadalo iterativní řešení?

- Použijeme zásobník, tak jako to standardně děláme pro "rozbalení" DFS
- Musíme ale dát pozor na pořadí operací: vybrat uzel ze zásobníku až když "ošetříme" následující uzly.

```

from collections import defaultdict, deque

class Graph:
    def __init__(self, vertices):
        self.graph = defaultdict(list)
        self.v = vertices

    def addEdge(self, u, v):
        self.graph[u].append(v)

    def topologicalSort(self):
        visited = [False] * self.v
        stack = deque()
        working_stack = deque()

        for i in range(self.v):
            if not visited[i]:
                working_stack.append(i)
                while working_stack:
                    # pop() až když zprocesujeme potomky
                    v = working_stack[-1]
                    visited[v] = True
                    daughters = [
                        next_v for next_v in self.graph[v]
                        if not visited[next_v]
                    ]

```

```

        if daughters:
            working_stack.extend(daughters)
        else:
            stack.appendleft(v)
            working_stack.pop()

    return stack

def main():
    g = Graph(6)
    g.addEdge(5, 2)
    g.addEdge(5, 0)
    g.addEdge(4, 0)
    g.addEdge(4, 1)
    g.addEdge(2, 3)
    g.addEdge(3, 1)

    print(g.topologicalSort())

if __name__ == "__main__":
    main()

```

Piškvorky

Toto je jednoduchá hra, a chceme najít optimální strategii. Kam dát následující kroužek?



Min-max strategie:

Data: seznam znaků x, o, . o délce 9 (nechceme 2D pole)

Hodnocení: Pokud se mřížce nachází trojice xxx, plus nekonečno. Pokud se v mřížce nachází trojice ooo, mínus nekonečno. Jinak 0.

Detekce: Pro každý znak najdeme všechna místa, kde se nachází, a porovnáme se seznamem 8 možných trojic:

```

INFINITY = 1
MINUS_INFINITY = - INFINITY

empty_grid = ["."] * 9
triples = [{0, 1, 2}, {3, 4, 5}, {6, 7, 8}, {0, 3, 6}, {1, 4, 7}, {2, 5, 8}, {0, 4, 8}, {2, 4, 6}]

def find_triple(grid, sign):
    positions = {i for i in range(9) if grid[i] == sign}
    result = [t for t in triples if t.issubset(positions)]
    return result

def grade(grid) -> int:
    if find_triple(grid, "x"):
        return INFINITY
    elif find_triple(grid, "o"):
        return MINUS_INFINITY
    else:
        return 0

def get_sign(player: bool) -> str:
    return "o" if player else "x"

```

Tisk mřížky:

```

def print_grid(grid) -> None:
    print()
    for i in range(3):
        for j in range(3):
            print(grid[3*i + j], end = " ")
        print()
    print(grade(grid))
    print()

```

Strom:

```

class Node:
    def __init__(self, grid):
        self.grid = grid
        self.df = self.grid.count(".")
        self.player = (9 - self.df) % 2
        self.score = grade(self.grid)
        self.children = []

```

Stavíme strom:

Musíme dát pozor na kombinatoriku. Mnohé pozice můžeme dosáhnout několika způsoby, takže pro pozici, kterou jsme již viděli, použijeme existující uzel stromu:

```

def build_tree(start_grid:list[int] = empty_grid) -> Node:
    node_dict = {}
    root = Node(start_grid)
    queue = deque([root])
    node_dict[tuple(start_grid)] = root
    n_nodes = 1
    while queue:
        node = queue.popleft()
        if node.score != 0:
            continue
        sign = get_sign(node.player)
        for pos in range(9):
            if node.grid[pos] == ".":
                new_grid = node.grid.copy()
                new_grid[pos] = sign
                if tuple(new_grid) in node_dict:
                    new_node = node_dict[tuple(new_grid)]
                else:
                    new_node = Node(new_grid)
                    node_dict[tuple(new_grid)] = new_node
                    queue.append(new_node)
                    n_nodes += 1
                node.children.append(new_node)
    print(n_nodes)
    return root

```

A konečně min-max:

```

class Choice:
    def __init__(self, choice, value):
        self.choice = choice
        self.value = value

def minmax(node):
    if not node.children:
        return Choice("end", node.score)

    choices = [minmax(c) for c in node.children]
    if node.player == 0:
        max_result = max(c.value for c in choices)
        max_choices = [i for i in range(len(node.children)) if choices[i].value ==
max_result]
        return Choice(max_choices, max_result)
    else:
        min_result = min(c.value for c in choices)
        min_choices = [i for i in range(len(node.children)) if choices[i].value ==
min_result]
        return Choice(min_choices, min_result)

```

```

def play(start_grid = empty_grid):
    tree = build_tree(start_grid)
    current_node = tree
    while True:
        print_grid(current_node.grid)
        choice = minmax(current_node)
        if choice.choice == "end":
            print("Game finished")
            break
        select = random.choice(choice.choice)
        current_node = current_node.children[select]

```

Výsledný program:

```

from collections import deque
import random

INFINITY = 1
MINUS_INFINITY = - INFINITY

empty_grid = ["."] * 9
triples = [{0, 1, 2}, {3, 4, 5}, {6, 7, 8}, {0, 3, 6}, {1, 4, 7}, {2, 5, 8}, {0, 4, 8}, {2, 4, 6}]

def find_triple(grid, sign):
    positions = {i for i in range(9) if grid[i] == sign}
    result = [t for t in triples if t.issubset(positions)]
    return result

def grade(grid) -> int:
    if find_triple(grid, "x"):
        return INFINITY
    elif find_triple(grid, "o"):
        return MINUS_INFINITY
    else:
        return 0

def get_sign(player: bool) -> str:
    return "o" if player else "x"

def print_grid(grid) -> None:
    print()
    for i in range(3):
        for j in range(3):
            print(grid[3*i + j], end = " ")
        print()
    print(grade(grid))
    print()

```

```

class Node:
    def __init__(self, grid):
        self.grid = grid
        self.df = self.grid.count(".")
        self.player = (9 - self.df) % 2
        self.score = grade(self.grid)
        self.children = []

def build_tree(start_grid:list[int] = empty_grid) -> Node:
    node_dict = {}
    root = Node(start_grid)
    queue = deque([root])
    node_dict[tuple(start_grid)] = root
    n_nodes = 1
    while queue:
        node = queue.popleft()
        if node.score != 0:
            continue
        sign = get_sign(node.player)
        for pos in range(9):
            if node.grid[pos] == ".":
                new_grid = node.grid.copy()
                new_grid[pos] = sign
                if tuple(new_grid) in node_dict:
                    new_node = node_dict[tuple(new_grid)]
                else:
                    new_node = Node(new_grid)
                    node_dict[tuple(new_grid)] = new_node
                    queue.append(new_node)
                    n_nodes += 1
                node.children.append(new_node)
    print(n_nodes)
    return root

class Choice:
    def __init__(self, choice, value):
        self.choice = choice
        self.value = value

    def __str__(self):
        return f"Choosing {self.choice} to reach {self.value}"

def minmax(node):
    if not node.children:
        return Choice("end", node.score)

    choices = [minmax(c) for c in node.children]
    if node.player == 0:

```

```

        max_result = max(c.value for c in choices)
        max_choices = [i for i in range(len(node.children)) if choices[i].value ==
max_result]
        return Choice(max_choices, max_result)
    else:
        min_result = min(c.value for c in choices)
        min_choices = [i for i in range(len(node.children)) if choices[i].value ==
min_result]
        return Choice(min_choices, min_result)

def play(start_grid = empty_grid):
    tree = build_tree(start_grid)
    current_node = tree
    while True:
        print_grid(current_node.grid)
        choice = minmax(current_node)
        if choice.choice == "end":
            print("Game finished")
            break
        select = random.choice(choice.choice)
        current_node = current_node.children[select]

def main() -> None:
    start_grid = input().split()
    play(start_grid)

if __name__ == "__main__":
    main()

```

Kód se chová v některých situacích divně:

```

. x x 0 . . . . .
200

. x x
0 . .
. . .
0

0 x x
0 . .
. . .
0

0 x x
0 x .
. . .

```

0

o x x

o x .

o . .

-1

Game finished

Process finished with exit code 0

To je způsobeno tím, že pozice, ze které vycházíme, je pro hráče * prohrávající a tedy všechno, co udělá, je stejně účinné. Abychom situaci trochu vylepšili, můžeme upřednostnit agresivní řešení - tedy cesty, které vedou k výhře rychleji:

```
def minmax(node):
    agresivity = 0.8
    if not node.children:
        return Choice("end", node.score)

    choices = [minmax(c) for c in node.children]
    if node.player == 0:
        max_result = max(c.value for c in choices)
        max_choices = [i for i in range(len(node.children)) if choices[i].value ==
max_result]
        return Choice(max_choices, agresivity * max_result)
    else:
        min_result = min(c.value for c in choices)
        min_choices = [i for i in range(len(node.children)) if choices[i].value ==
min_result]
        return Choice(min_choices, agresivity * min_result)
```

Domácí úkoly

1. **Bipartitní graf:** zjistěte, zda je možné obarvit vrcholy grafu dvěma barvami tak, aby žádné dva sousedící vrcholy neměli stejnou barvu.
2. **Lidské kruhy :** N lidí se náhodně pochyta za ruce. Máte zjistit, kolik kruhů vytvoří.